

An agent harness , also known as agent scaffolding, is the software infrastructure surrounding a large language model (LLM) that enables it to operate as an AI agent. It manages tool use, memory, state persistence, execution environments and feedback loops, as opposed to the model's internal reasoning. The UK's AI Security Institute described an AI agent as being formed by the model plus the scaffolding back in 2023. The relationship can be expressed as $\text{Agent} = \text{Model} + \text{Harness}$.

Because an LLM is stateless and, unaided, produces only text, the harness is what allows a model to take actions over multiple steps, use external tools, and sustain a long-running task across sessions. Rather than repeatedly re-reading an ever-growing transcript inside the context window, a harness can offload record-keeping into a structured software environment that manages the agent's state. A minimal harness is unnecessary for a single prompt-and-response exchange, but becomes important as tasks grow multi-step, tool-oriented, or long-running.

Architecture

The mechanisms that a harness coordinates predate the term. The pattern of a model alternating between reasoning and acting in a loop was introduced in the peer-reviewed ReAct framework, and the ability of a model to call external tools was demonstrated in Toolformer. Components commonly grouped under a harness include tool dispatch, memory and state persistence, an isolated sandbox or workspace, context management, and guardrails such as scoped permissions, approval tiers and monitoring. In a 2026 engineering write-up, Anthropic described a worked example built from an initialiser agent that prepares the environment and a coding agent that repeatedly selects the next unfinished task, commits, and updates a progress record before stopping.

Birgitta Böckeler of Thoughtworks distinguishes an inner harness shipped by the model's builder (for example an agent SDK or a coding tool such as Cursor or Codex) from an outer

harness that the user assembles on top, such as instruction files, Model Context Protocol servers and custom skills. She further separates guides, which steer the agent before it acts, from sensors, which observe the result and let the agent self-correct; each may be computational (deterministic checks such as linters or tests) or inferential (semantic checks such as an "LLM as judge").

History and terminology

Related notions of scaffolding around a model exist in software testing (the test harness), in LLM benchmarking (evaluation harnesses), and, in reinforcement learning, in the environment and wrapper that surround a learning agent. The nouns agent harness and LLM harness circulated among practitioners before the associated discipline was named.

The vocabulary of "harness engineering" emerged in early 2026, and attribution of the specific phrase is contested. Several accounts trace it to a February 2026 blog post by Mitchell Hashimoto, co-founder of HashiCorp, who described a practice of engineering a permanent fix into an agent's environment each time it makes a mistake; other accounts credit Vivek Trivedy of LangChain, whose "Anatomy of an Agent Harness" post derived the harness components from the Agent = Model + Harness formula. A widely cited OpenAI engineering report describing a large codebase built largely by coding agents, together with follow-on writing from Thoughtworks, LangChain and Anthropic, helped spread the term.

By mid-2026 the harness had become an object of academic study. Independent reporting described research such as Self-Harness, in which an agent iteratively mines its own failures to propose and validate changes to its harness, and Harness-1, an open-source search agent that improved retrieval accuracy chiefly by redesigning the software environment around the model rather than by enlarging the model.

Relationship to prompt and context engineering

Harness engineering is often positioned as a broader layer than prompt engineering, which optimises a single interaction, or context engineering, which governs what information the model sees at a given moment; in this framing the harness designs the whole operational environment and contains the other two as parts. A distinguishing feature is that the component being wrapped is non-deterministic, so a harness is designed to recover gracefully when the model fabricates an action or reports a task as finished when it is not.

See also

AI agent

Large language model

Prompt engineering

Model Context Protocol

Software agent

Test harness

== References ==